PROJECT BASED LEARNING

CONTAINER RESOURCE ALLOCATION SIMULATOR

Operating Systems Lab (CIC-353)

Computer Science & Engineering

Session: 2026-2027

PRESENTED BY

Vedang Goel 12518002724

Samar Jaryal 15518002724

Soumya Madhyan 15618002724

Under the Guidance of: Dr.Yashu Shanker

CONTENT

01 **Introduction**
Problem, need and objectives

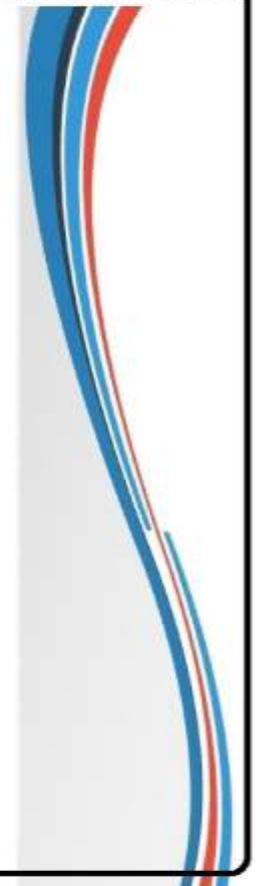
02 **Methodology & Architecture**
Simulation cycle and components

03 **CPU & Memory Models**
Allocation, limits and isolation

04 **Implementation & Results**
Dashboard and scenario evaluation

05 **Conclusion & References**
Scope, future work and sources





INTRODUCTION

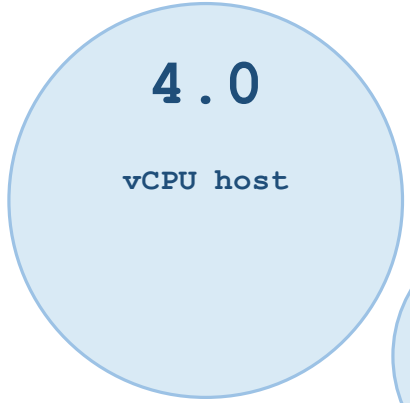
A shared host needs fair CPU allocation and clear memory boundaries

Problem

Multiple containers share finite CPU and RAM. A busy workload can slow other services or exceed its permitted memory.

Why it matters

- Makes resource contention visible
- Shows CPU throttling and weighted sharing
- Demonstrates per-container memory isolation





CPU limits

RAM limits

FEASIBILITY & OBJECTIVES

A Python-based simulator that can run on a standard laptop

Technical	Economic	Operational
Python 3; modular simulation logic	Free tools; no paid platform required	Configure, simulate and observe

Objectives

01 Model host
CPU and RAM capacity

02 Set container limits
Demand, cap, weight, memory

03 Simulate contention
Weighted sharing and throttling

04 Monitor isolation
Memory alerts and event history

RESOURCE MANAGEMENT BASICS

CPU and memory require different allocation controls

CPU

REQUEST

CPU demand

How much processing capacity a workload requests. Demand can exceed the host capacity when several containers run together.

W

PRIORITY

CPU cap & weight

A cap limits the maximum CPU a container may receive. A relative weight determines its share during contention.

RAM

BOUNDARY

Memory hard limit

A per-container boundary. Usage above the limit triggers a memory-pressure warning or simulated OOM event.

CPU controls affect scheduling and sharing; memory limits define an individual usage boundary.

CONTAINER RESOURCE MODEL

Three workloads compete inside one simulated host

HOST SYSTEM | 4.0 vCPU | 4096 MB RAM

WEB

CPU demand 2.4 | cap 2.0 | weight 2

Memory 700 / 1024 MB

DATABASE

CPU demand 2.5 | cap 2.0 | weight 2

Memory 1800 / 2048 MB

WORKER

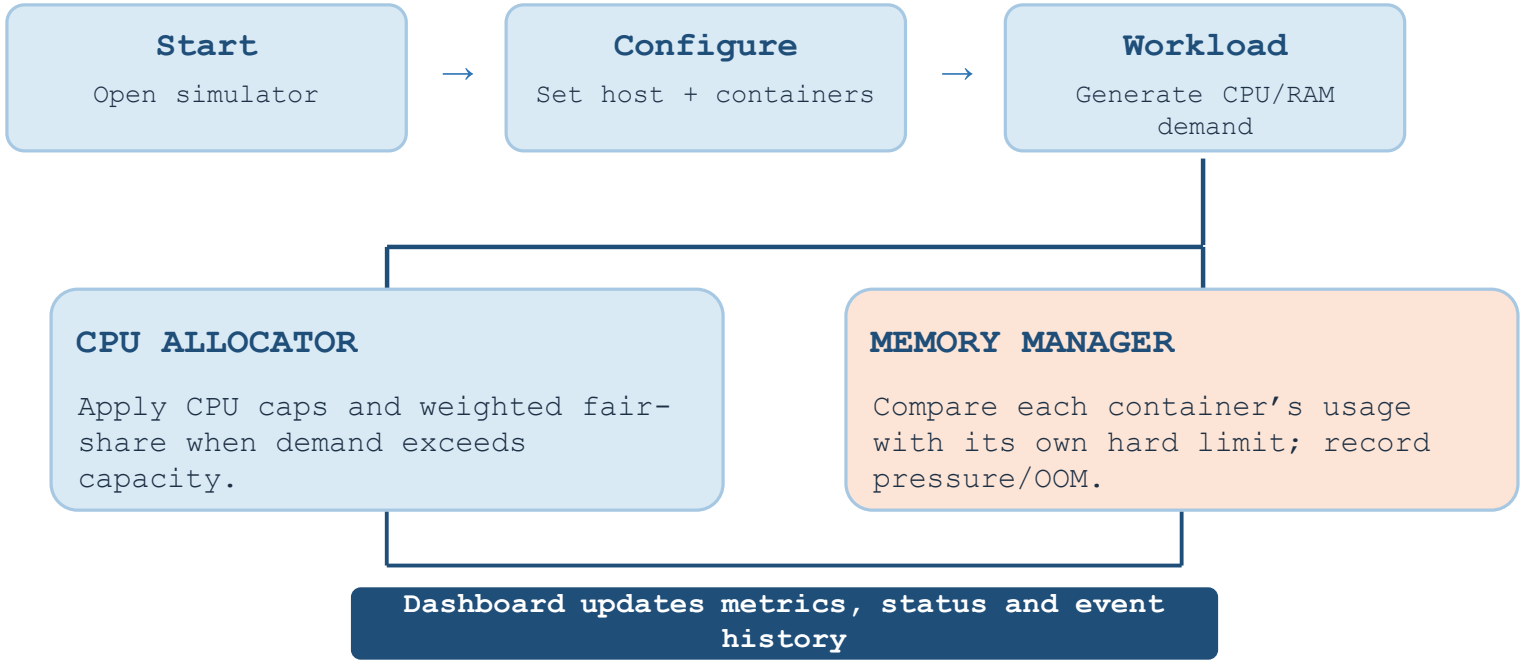
CPU demand 1.5 | cap 1.0 | weight 1

Memory 1150 / 1024 MB

Each container keeps its own CPU and memory counters; CPU allocations must remain within host capacity.

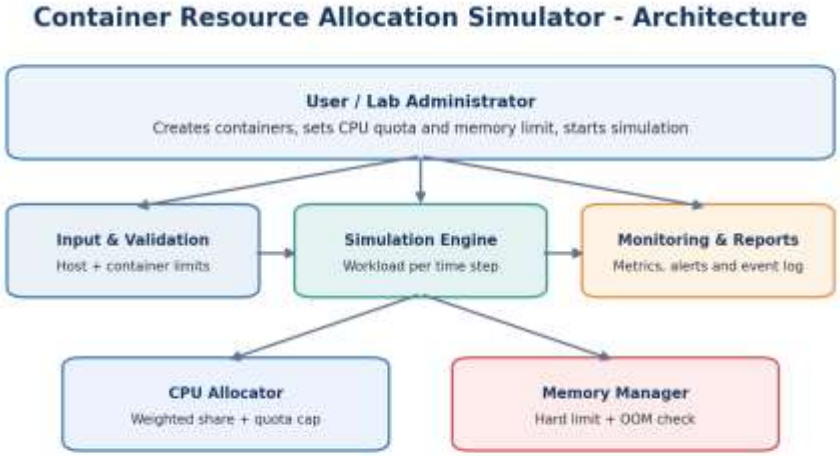
SIMULATION FLOWCHART

Each run applies CPU allocation and memory checks independently



SYSTEM ARCHITECTURE

Five modules form one repeatable resource-simulation cycle



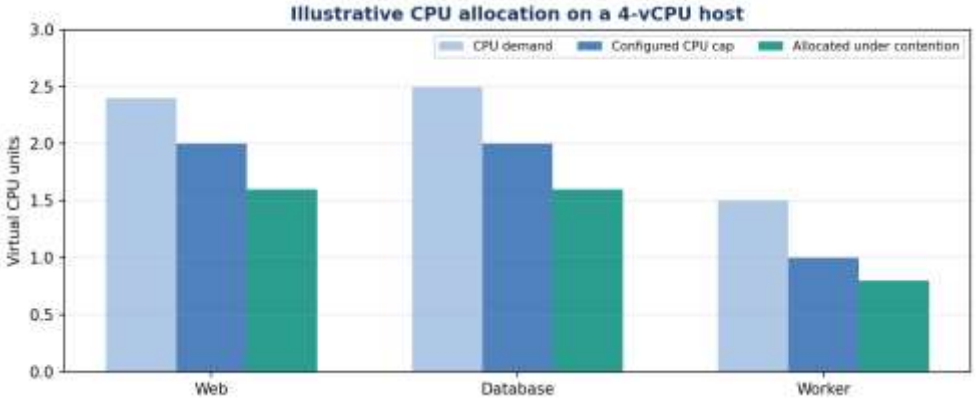
Main components

- Input: host and container settings
- Engine: workload per time step
- CPU allocator: caps + weights
- Memory manager: hard-limit checks
- Monitoring: metrics, alerts and logs

Data flow: Configuration → Workload → CPU / Memory checks → Dashboard

CPU ALLOCATION MODEL

Weighted sharing during contention, bounded by each CPU cap



Allocation rule

Eligible demand: $e_i = \min(d_i, q_i)$

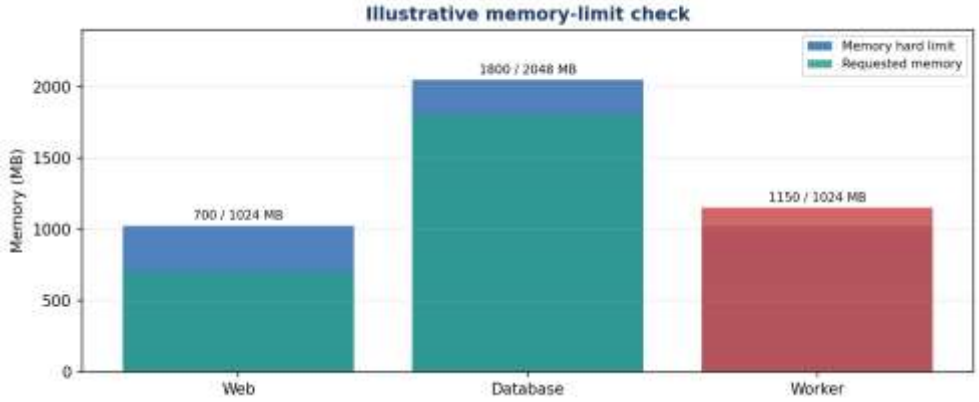
If total eligible demand exceeds host capacity, available CPU is shared in proportion to active weights. Unused share is redistributed.

$\Sigma a_i \leq C$ and $0 \leq a_i \leq e_i$

Example on 4 vCPU: Web 1.6 + Database 1.6 + Worker 0.8 = 4.0 vCPU allocated.

MEMORY LIMIT & ISOLATION

A hard limit is a boundary, not a weighted share



Memory check

Web: 700 / 1024 MB
Database: 1800 / 2048 MB
Worker: 1150 / 1024 MB

Worker exceeds its limit by 126 MB and is flagged for memory pressure.

OOM is simulated; no real process is terminated.

Separate memory counters prevent one container's usage from overwriting another's.

SIMULATOR IMPLEMENTATION

Configure containers, run a scenario and inspect the dashboard

1. **Host setup**
   CPU capacity and total RAM
2. **Container setup**
   Demand, cap, weight, memory limit
3. **Run workload**
   Normal, contention or pressure
4. **Review output**
   Allocation, warnings and event log



Illustrative dashboard mock-up — not a live execution screenshot

Modules: Host Configuration • Container Manager • Workload Generator • CPU / Memory Manager • Dashboard

RESULTS & DISCUSSION

Illustrative 4-vCPU host with 3 containers under resource contention

Container	CPU demand / cap	Weight	CPU alloc.	Memory use / limit (MB)	Status
Web	2.4 / 2.0	2	1.6	700 / 1024	Normal
Database	2.5 / 2.0	2	1.6	1800 / 2048	Normal
Worker	1.5 / 1.0	1	0.8	1150 / 1024	Memory pressure

CPU contention

Eligible demand is 5.0 vCPU; the host has 4.0 vCPU. Weighted allocation fills the available capacity.

Memory pressure

Worker exceeds its 1024 MB limit by 126 MB; other containers remain within their own limits.

Values are illustrative calculations.

SCOPE OF THE PROJECT

A learning simulator for resource controls and accounting isolation

What it demonstrates

- Host CPU and RAM configuration
- Per-container CPU caps and weights
- Weighted CPU sharing during contention
- Memory hard limits and pressure alerts
- Separate resource counters and event history

Current limitations

- Does not start real Docker containers
- Does not change Linux kernel settings
- CPU model is simplified weighted sharing
- OOM is represented as a simulated state
- Results are not physical host benchmarks

Real Linux cgroups provide kernel-level resource accounting and enforcement.

CONCLUSION & FUTURE WORK

CPU and memory isolation need separate but coordinated controls

Conclusion

- Weighted sharing distributes CPU under contention
- CPU caps bound each container's allocation
- Memory hard limits detect overuse
- Separate counters demonstrate accounting isolation

Future Work

- Integrate Docker Engine and Linux cgroup v2
- Add time-series charts and downloadable logs
- Compare CPU scheduling policies
- Add memory reservation, swap and OOM options

REFERENCES

Documentation and textbooks used for the resource-management concepts

[1] Docker Documentation, “Resource constraints,” Docker Engine Manual. docs.docker.com/engine/containers/resource_constraints/

[2] The Linux Kernel Documentation, “Control Group v2.” kernel.org/doc/html/latest/admin-guide/cgroup-v2.html

[3] Kubernetes Documentation, “Resource Management for Pods and Containers.” kubernetes.io/docs/concepts/configuration/manage-resources-containers/

[4] A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018.

[5] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed., Pearson, 2015.

[6] Python Software Foundation, Python 3 Documentation and unittest module. docs.python.org/3/